B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution, Affiliated to VTU, Belagavi)

Bull Temple Road, Basavanagudi, Bengaluru – 560 019

Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

Harshitha U S – 1BM24EC072

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Prof. Ajaykumar Devarapalli** for his valuable guidance, encouragement, and support throughout the completion of this Alternative Assessment Tool (AAT) on "Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

We extend our thanks to the **Department of Electronics and Communication Engineering, BMS College of Engineering**, for providing the necessary facilities and resources to carry out this work.

We are also thankful to our faculty members and everyone who directly or indirectly supported us in completing this report successfully.

Finally, we thank our team members for their cooperation and contribution towards the successful completion of this AAT.

Abstract

This report presents the design and behavioral simulation of digital modules using Verilog HDL and Xilinx Vivado. The modules include an arithmetic logic unit, comparator, parity generator and checker, barrel shifter, timer, ring and Johnson counters, synchronous and asynchronous counters, shift register, divide-by-N counter, Mealy and Moore finite-state machines, and a pseudo-random sequence generator. Each experiment includes the aim, design description, Verilog source code, testbench, and reserved spaces for the circuit/RTL schematic and simulation output screenshots.

Software and Tools

Software: Xilinx Vivado Design Suite (behavioral simulation using xsim). Language: Verilog HDL. Simulation is performed using a testbench that applies input sequences and observes outputs in the waveform window and Tcl Console.

General Procedure in Vivado

- Create a new RTL project in Vivado and select the FPGA part/board if required.
- Add the relevant design .v file under Design Sources.
- Add its matching testbench .v file under Simulation Sources.
- Right-click the testbench module and select Set as Top.
- Select Run Simulation → Run Behavioral Simulation.
- Inspect the waveform and Tcl Console. Add internal or output signals to the waveform if they are not displayed.
- Capture the RTL schematic (where available) and waveform/output, then paste the screenshots in the spaces provided in each experiment.
- Repeat for every module, running one testbench at a time.

1. Arithmetic Logic Unit (ALU)

Aim

Design a 4-bit ALU that performs AND, OR, XOR, addition, subtraction, and pass-through operations selected by a 3-bit control input.

Design Description

Inputs: A[3:0], B[3:0], sel[2:0]. Output: Y[3:0], carry/borrow flag C. Operation map: 000=A AND B; 001=A OR B; 010=A XOR B; 011=A+B; 100=A−B; 101=A; 110=B; 111=0. For arithmetic operations, C is the carry-out for addition and the borrow indicator for subtraction (implemented as A < B).

Control truth table (A and B are 4-bit inputs):

sel	Operation
000	A AND B
001	A OR B
010	A XOR B
011	A + B
100	A − B
101	Pass A
110	Pass B
111	Output 0000

Verilog HDL Design Code

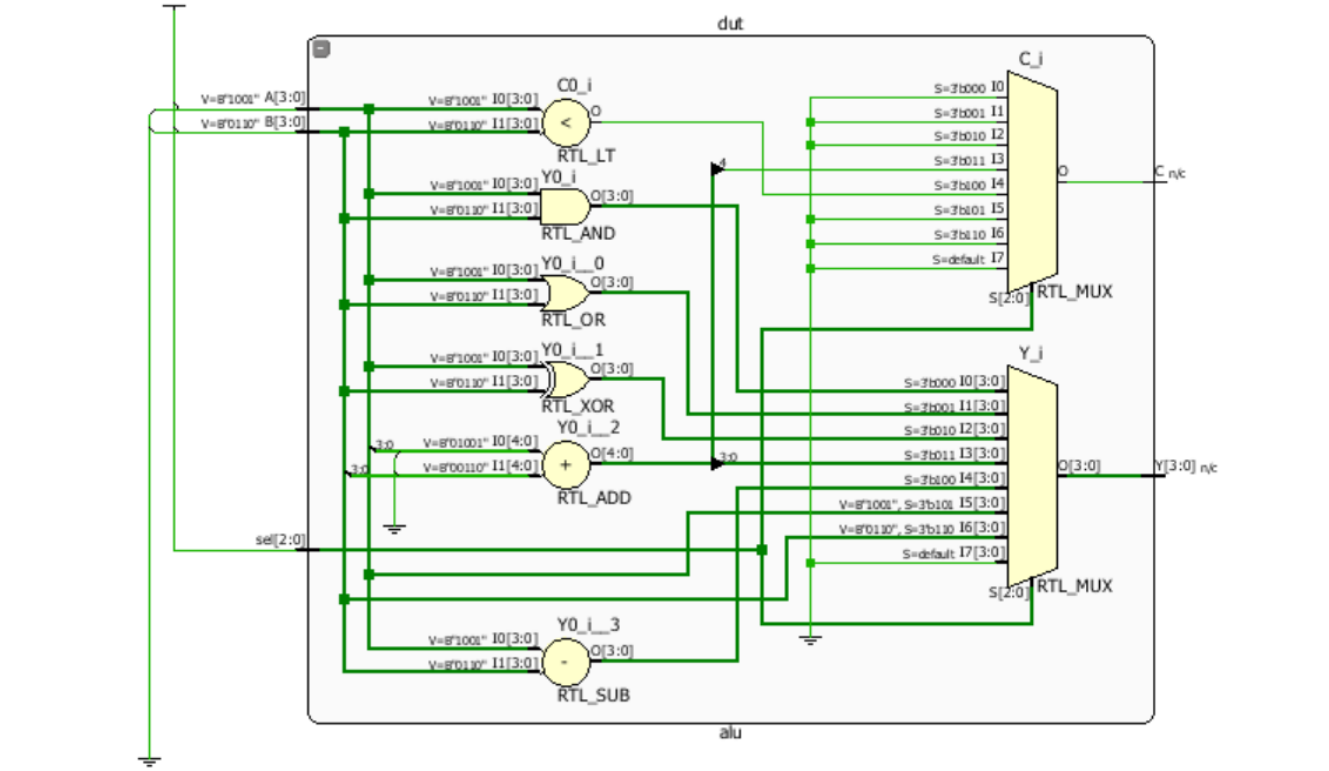
```
module alu4 (
   input  wire [3:0] A, B,
   input  wire [2:0] sel,
   output reg  [3:0] Y,
   output reg        C
);
always @* begin
   Y = 4'b0000;
   C = 1'b0;
   case (sel)
      3'b000: Y = A & B;
      3'b001: Y = A | B;
      3'b010: Y = A ^ B;
      3'b011: {C, Y} = {1'b0, A} + {1'b0, B};
      3'b100: begin Y = A - B; C = (A < B); end
      3'b101: Y = A;
```

```
        3'b110: Y = B;
        default: begin Y = 4'b0000; C = 1'b0; end
    endcase
end
endmodule
```

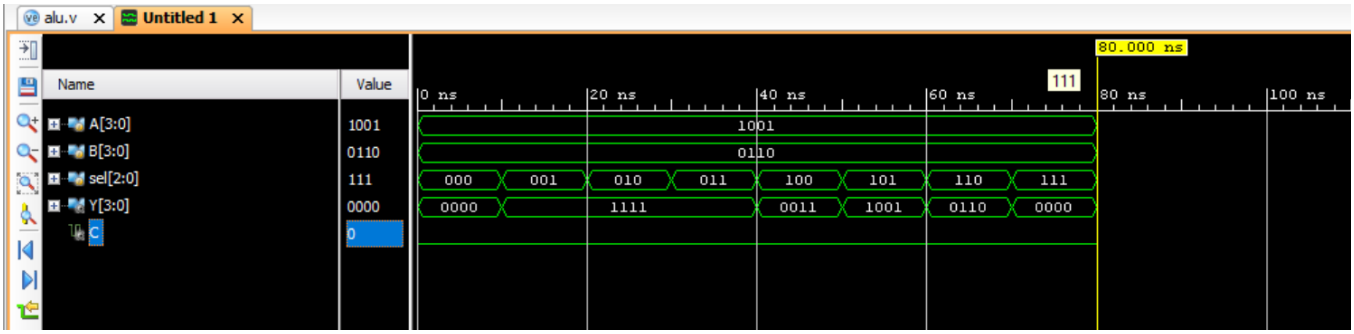
Testbench (TB)

```
`timescale 1ns/1ps
module tb_alu4;
reg [3:0] A, B;
reg [2:0] sel;
wire [3:0] Y;
wire C;
alu4 dut(.A(A), .B(B), .sel(sel), .Y(Y), .C(C));
initial begin
    A=4'd9; B=4'd6;
    sel=3'b000; #10; $display("AND: A=%d B=%d Y=%d C=%b",A,B,Y,C);
    sel=3'b001; #10; $display("OR : Y=%d",Y);
    sel=3'b010; #10; $display("XOR: Y=%d",Y);
    sel=3'b011; #10; $display("ADD: Y=%d C=%b",Y,C);
    sel=3'b100; #10; $display("SUB: Y=%d Borrow=%b",Y,C);
    sel=3'b101; #10; $display("PASS A: Y=%d",Y);
    sel=3'b110; #10; $display("PASS B: Y=%d",Y);
    sel=3'b111; #10; $display("ZERO: Y=%d",Y);
    $finish;
end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



2. Comparator

Aim

Compare two unsigned 4-bit values and indicate whether A is greater than, equal to, or less than B.

Design Description

Inputs: A[3:0], B[3:0]. Outputs: gt, eq, lt. Exactly one output is high for any input pair.

Comparator truth table:

Condition	gt	eq	lt
A > B	1	0	0
A = B	0	1	0
A < B	0	0	1

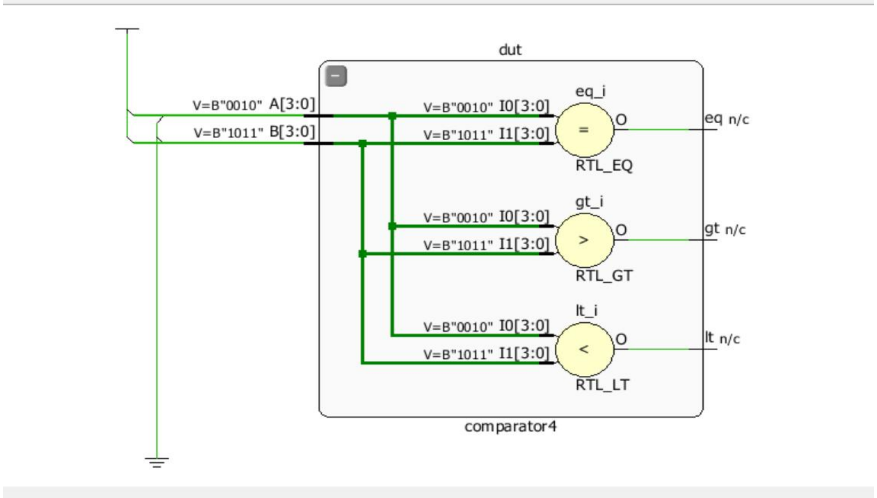
Verilog HDL Design Code

```
module comparator4 (
   input wire [3:0] A, B,
   output wire gt, eq, lt
);
assign gt = (A > B);
assign eq = (A == B);
assign lt = (A < B);
endmodule
```

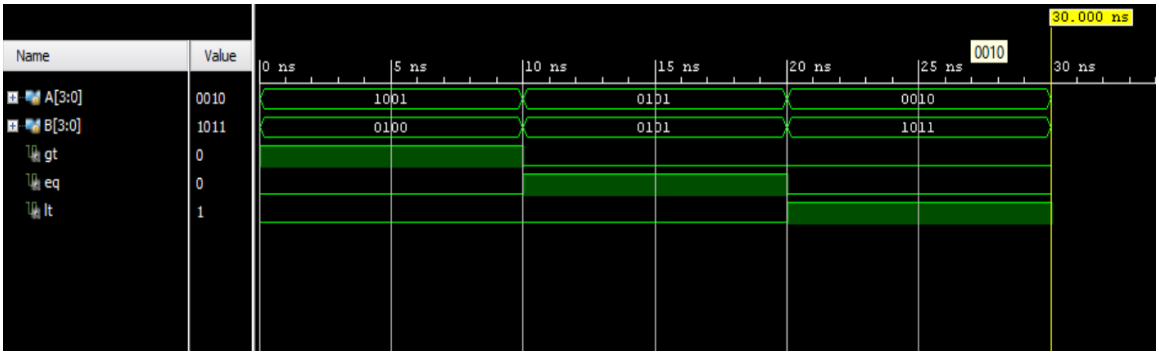
Testbench (TB)

```
`timescale 1ns/1ps
module tb_comparator4;
reg [3:0] A, B;
wire gt, eq, lt;
comparator4 dut(.A(A), .B(B), .gt(gt), .eq(eq), .lt(lt));
initial begin
   A=4'd9; B=4'd4; #10; $display("A>B: gt=%b eq=%b lt=%b",gt,eq,lt);
   A=4'd5; B=4'd5; #10; $display("A=B: gt=%b eq=%b lt=%b",gt,eq,lt);
   A=4'd2; B=4'd11; #10; $display("A<B: gt=%b eq=%b lt=%b",gt,eq,lt);
   $finish;
end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



3. 4-Bit Parity Generator and Checker

Aim

Generate an even-parity bit for a 4-bit data word and check whether the received 5-bit word has valid even parity.

Design Description

The generator output parity = A[3]^A[2]^A[1]^A[0], so the XOR of data and parity is zero. The checker takes data[3:0] and parity_in; error=1 indicates odd parity across the received word.

Even-parity truth table (P = XOR of all data bits):

Number of 1s in data	Generated P	Total parity after adding P
Even	0	Even
Odd	1	Even

Verilog HDL Design Code

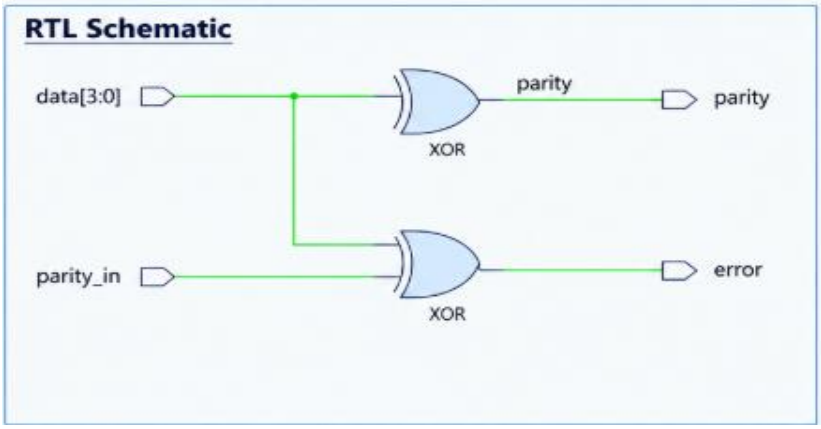
```
module parity4 (
   input wire [3:0] data,
   output wire parity,
   input wire parity_in,
   output wire error
);
assign parity = ^data;            // even-parity bit to append
assign error  = ^{data, parity_in}; // 1 means parity error
endmodule
```

Testbench (TB)

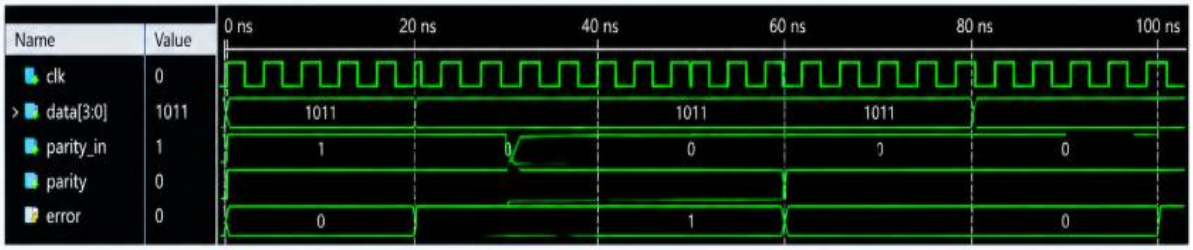
```
`timescale 1ns/1ps
module tb_parity4;
reg [3:0] data;
reg parity_in;
wire parity, error;
parity4 dut(.data(data), .parity(parity), .parity_in(parity_in), .error(error));
initial begin
   data=4'b1011; #10;
   parity_in=parity; #10;
   $display("data=%b generated parity=%b received parity=%b error=%b",data,parity,parity_in,error);
   parity_in=~parity; #10;
   $display("Injected parity error: error=%b",error);
   data=4'b0000; #10; parity_in=1'b0; #10;
```

```
    $display("data=%b parity=%b error=%b",data,parity,error);
    $finish;
end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



4. 4-Bit Barrel Shifter

Aim

Shift a 4-bit input left or right by 0–3 bit positions in one combinational operation.

Design Description

Inputs: data[3:0], shamt[1:0], dir. dir=0 shifts left; dir=1 shifts right. Vacated positions are filled with zeros; shifted-out bits are discarded.

Shift control truth table:

dir	shamt	Operation
0	00	Shift left by 0
0	01	Shift left by 1
0	10	Shift left by 2
0	11	Shift left by 3
1	00	Shift right by 0
1	01	Shift right by 1
1	10	Shift right by 2
1	11	Shift right by 3

Verilog HDL Design Code

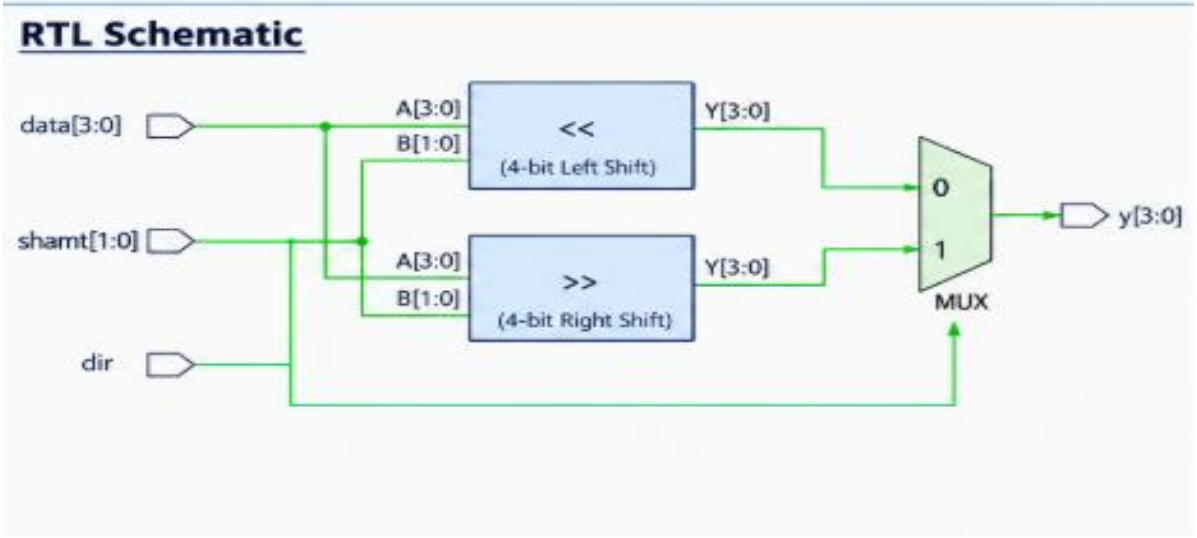
```
module barrel_shifter4 (
   input wire [3:0] data,
   input wire [1:0] shamt,
   input wire dir,
   output reg [3:0] y
);
always @* begin
   if (dir == 1'b0)
      y = data << shamt;
   else
      y = data >> shamt;
end
endmodule
```

Testbench (TB)

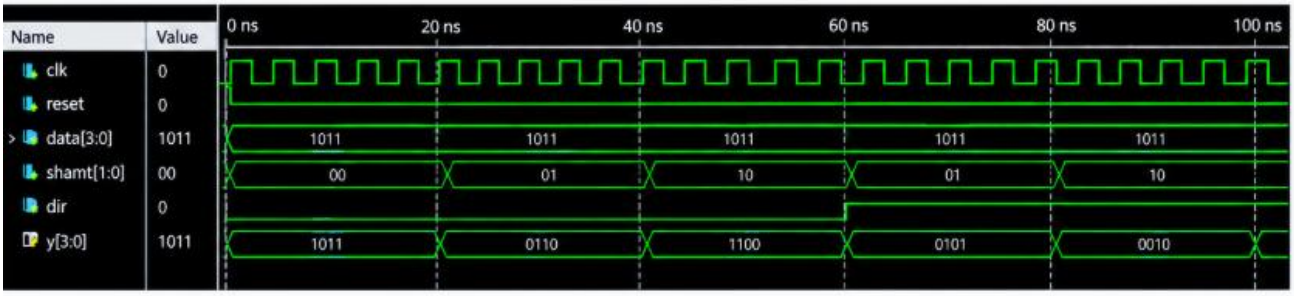
```
`timescale 1ns/1ps
module tb_barrel_shifter4;
reg [3:0] data;
reg [1:0] shamt;
reg dir;
```

```
wire [3:0] y;
barrel_shifter4 dut(.data(data), .shamt(shamt), .dir(dir), .y(y));
initial begin
   data=4'b1011; dir=0;
   shamt=0; #10; $display("left shift 0: %b",y);
   shamt=1; #10; $display("left shift 1: %b",y);
   shamt=2; #10; $display("left shift 2: %b",y);
   dir=1; shamt=1; #10; $display("right shift 1: %b",y);
   shamt=2; #10; $display("right shift 2: %b",y);
   $finish;
end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



5. Timer Module

Aim

Create a parameterized clocked timer that counts clock cycles and produces a one-clock-cycle pulse at the programmed terminal count.

Design Description

Inputs: clk, reset (active high), enable. Parameter PERIOD sets the number of enabled clock cycles per pulse. pulse is high for one cycle when the counter reaches PERIOD−1. Set PERIOD to a small value in simulation.

Timer control table:

reset	enable	Behavior
1	X	Counter cleared; pulse=0
0	0	Counter holds; pulse=0
0	1	Counts; pulse at terminal count

Verilog HDL Design Code

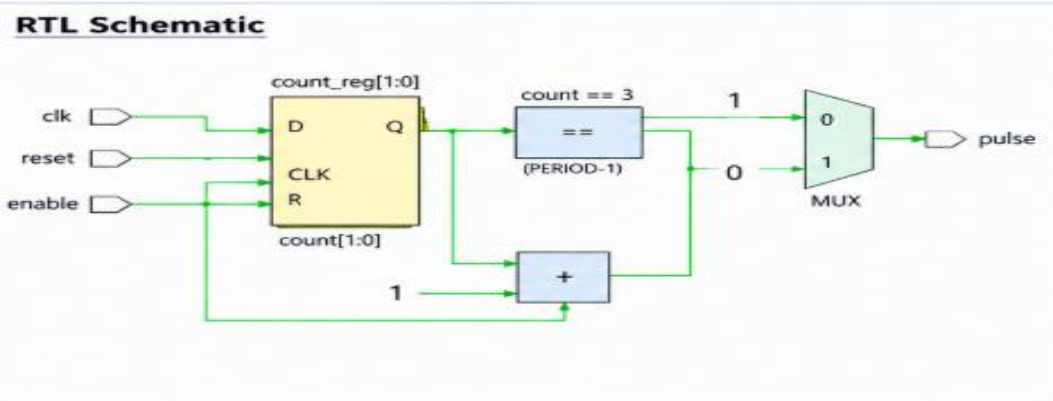
```
module timer_module #(
   parameter PERIOD = 10
) (
   input wire clk, reset, enable,
   output reg pulse
);
localparam CW = (PERIOD <= 1) ? 1 : $clog2(PERIOD);
reg [CW-1:0] count;
always @(posedge clk) begin
   if (reset) begin
      count <= 0;
      pulse <= 0;
   end else if (enable) begin
      if (count == PERIOD-1) begin
         count <= 0;
         pulse <= 1;
      end else begin
         count <= count + 1'b1;
         pulse <= 0;
      end
   end else begin
      pulse <= 0;
   end
```

```
end
endmodule
```

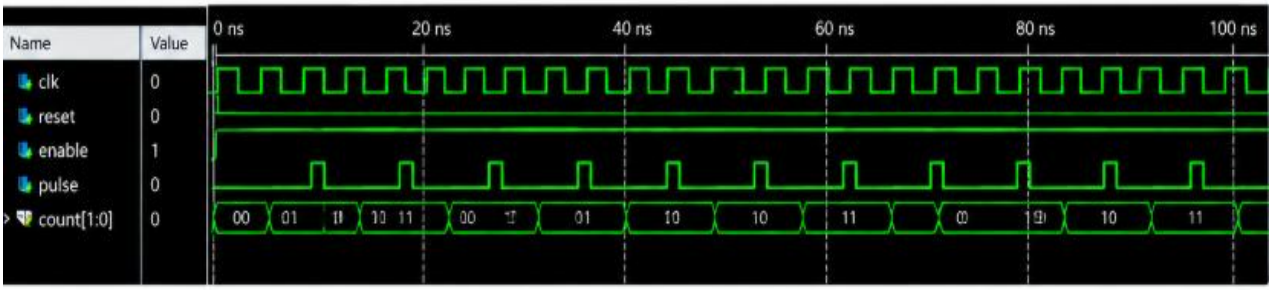
Testbench (TB)

```
`timescale 1ns/1ps
module tb_timer_module;
reg clk=0, reset=1, enable=0;
wire pulse;
always #5 clk=~clk;
timer_module #(.PERIOD(4)) dut(.clk(clk),.reset(reset),.enable(enable),.pulse(pulse));
initial begin
   #12; reset=0; enable=1;
   repeat (12) begin
      @(posedge clk); #1; $display("time=%0t pulse=%b",$time,pulse);
   end
   $finish;
end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



6. Ring Counter

Aim

Implement a 4-bit ring counter in which a single logic 1 circulates through the register.

Design Description

On reset the state is 0001. On each rising clock edge the pattern rotates left: 0001, 0010, 0100, 1000, then repeats. This is a one-hot sequence.

State transition table:

Present state	Next state
0001	0010
0010	0100
0100	1000
1000	0001

Verilog HDL Design Code

```
module ring_counter4 (
   input wire clk, reset,
   output reg [3:0] q
);
always @(posedge clk) begin
   if (reset)
      q <= 4'b0001;
   else
      q <= {q[2:0], q[3]};
end
endmodule
```

Testbench (TB)

```
module ring_counter_tb;
   reg clk = 0, rst = 1;
   wire [3:0] q;

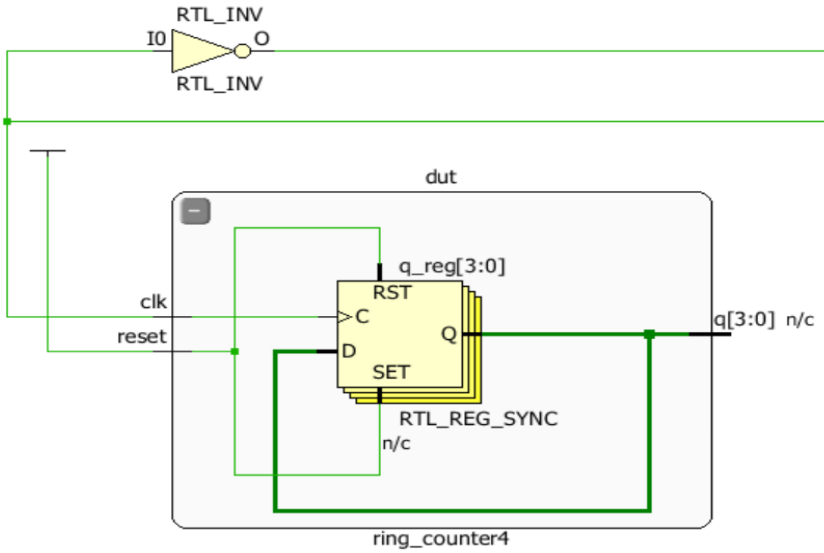
   ring_counter dut (.clk(clk), .rst(rst), .q(q));

   always #5 clk = ~clk;

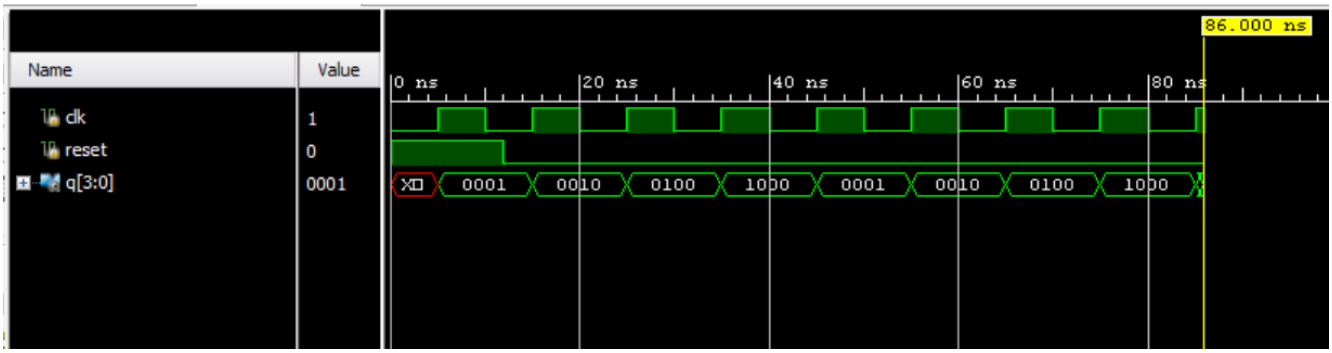
   initial begin
      $monitor("t=%0t rst=%b q=%b", $time, rst, q);
      #12 rst = 0;
```

```
        #100 $finish;
    end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



7. Johnson Counter

Aim

Implement a 4-bit Johnson counter that cycles through eight distinct states.

Design Description

On each rising edge, shift the register and feed the inverted most-significant bit back to the least-significant bit. Reset state is 0000.

State transition table:

Present state	Next state
0000	0001
0001	0011
0011	0111
0111	1111
1111	1110
1110	1100
1100	1000
1000	0000

Verilog HDL Design Code

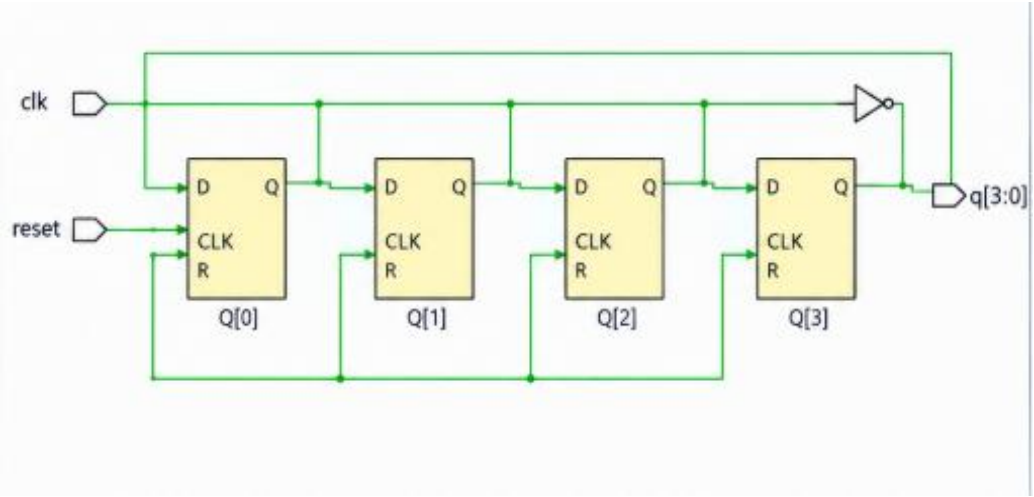
```
module johnson_counter4 (
   input wire clk, reset,
   output reg [3:0] q
);
always @(posedge clk) begin
   if (reset)
      q <= 4'b0000;
   else
      q <= {q[2:0], ~q[3]};
end
endmodule
```

Testbench (TB)

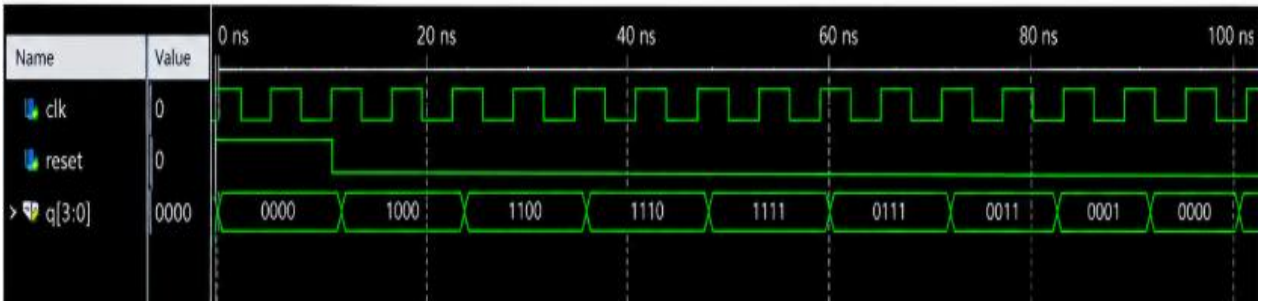
```
`timescale 1ns/1ps
module tb_johnson_counter4;
reg clk=0, reset=1;
wire [3:0] q;
always #5 clk=~clk;
johnson_counter4 dut(.clk(clk),.reset(reset),.q(q));
initial begin
```

```
    #12; reset=0;
    repeat (10) begin @(posedge clk); #1; $display("time=%0t q=%b",$time,q); end
    $finish;
end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



8. Synchronous and Asynchronous Counters

Aim

Implement and compare a 4-bit synchronous binary up-counter and a 4-bit asynchronous ripple counter.

Design Description

The synchronous counter updates all bits on the same rising edge of clk. The ripple counter uses each preceding bit as the next stage clock and is intended for illustrating asynchronous propagation. The asynchronous design is simulation/teaching-oriented; use suitable clocking and reset strategy for hardware.

Counter behavior table:

Counter type	Clocking	State sequence
Synchronous	All bits use the same clock edge	0000 → 0001 → … → 1111 → 0000
Asynchronous (ripple)	Each stage is clocked by the preceding bit	Binary count; transitions ripple between stages

Verilog HDL Design Code

```
module sync_counter4 (
   input wire clk, reset,
   output reg [3:0] q
);
always @(posedge clk) begin
   if (reset) q <= 4'b0000;
   else       q <= q + 1'b1;
end
endmodule

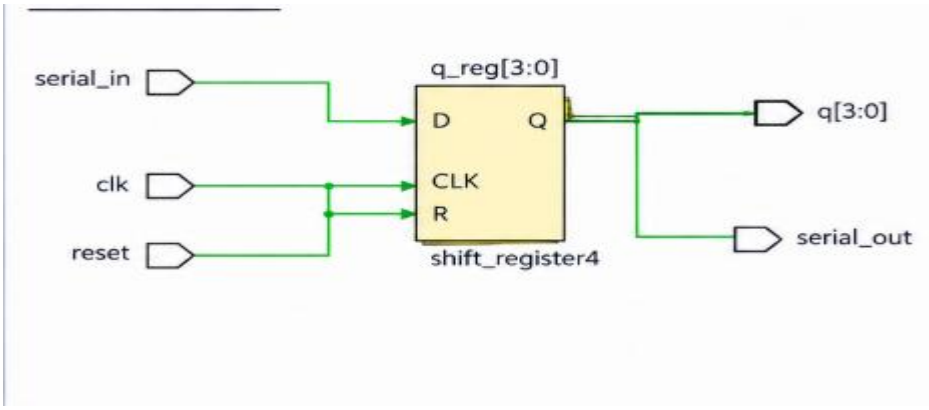
module async_counter4 (
   input wire clk, reset,
   output reg [3:0] q
);
always @(negedge clk or posedge reset) begin
   if (reset) q[0] <= 1'b0;
   else       q[0] <= ~q[0];
end
always @(negedge q[0] or posedge reset) begin
   if (reset) q[1] <= 1'b0;
   else       q[1] <= ~q[1];
```

```
end
always @(negedge q[1] or posedge reset) begin
   if (reset) q[2] <= 1'b0;
   else       q[2] <= ~q[2];
end
always @(negedge q[2] or posedge reset) begin
   if (reset) q[3] <= 1'b0;
   else       q[3] <= ~q[3];
end
endmodule
```

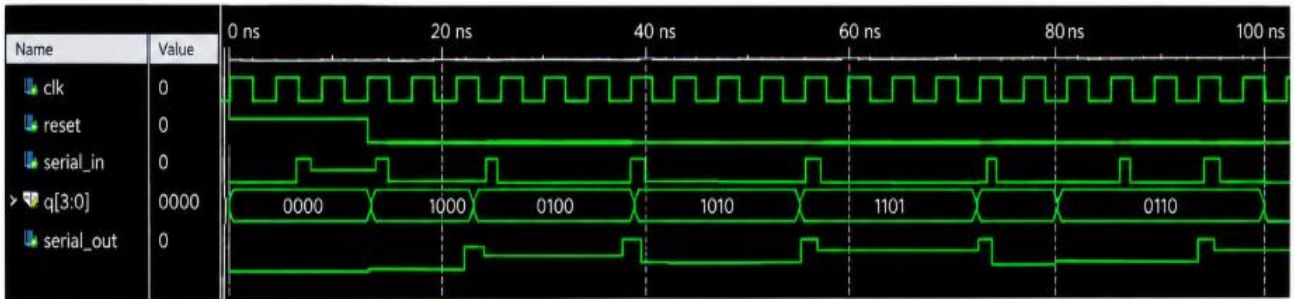
Testbench (TB)

```
`timescale 1ns/1ps
module tb_counters;
reg clk=0, reset=1;
wire [3:0] qs, qa;
always #5 clk=~clk;
sync_counter4 s0(.clk(clk),.reset(reset),.q(qs));
async_counter4 a0(.clk(clk),.reset(reset),.q(qa));
initial begin
   #12; reset=0;
   repeat (18) begin
      @(posedge clk); #2;
      $display("time=%0t sync=%b async=%b",$time,qs,qa);
   end
   $finish;
end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



9. Shift Register

Aim

Design a 4-bit serial-in, serial-out shift register with serial input and serial output.

Design Description

On each rising clock edge, serial_in enters the most-significant bit and the previous bits shift toward the least-significant bit. q[0] is the serial output.

Shift register operation table:

serial_in	Next q[3:0]
0	{0, q[3:1]}
1	{1, q[3:1]}

Verilog HDL Design Code

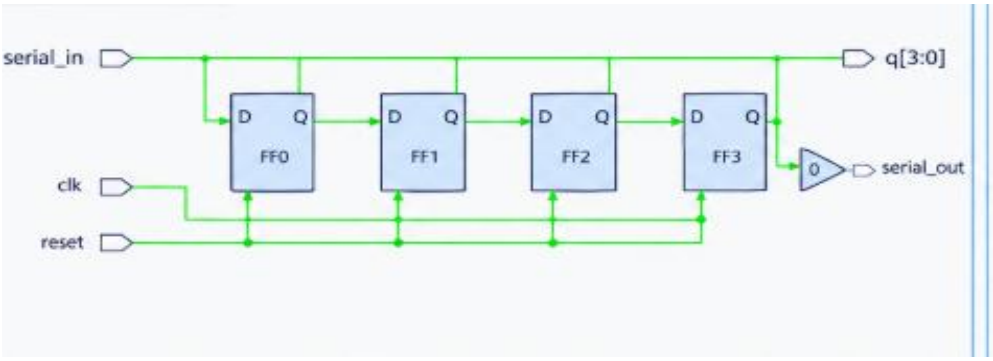
```
module shift_register4 (
   input wire clk, reset, serial_in,
   output reg [3:0] q,
   output wire serial_out
);
assign serial_out = q[0];
always @(posedge clk) begin
   if (reset) q <= 4'b0000;
   else       q <= {serial_in, q[3:1]};
end
endmodule
```

Testbench (TB)

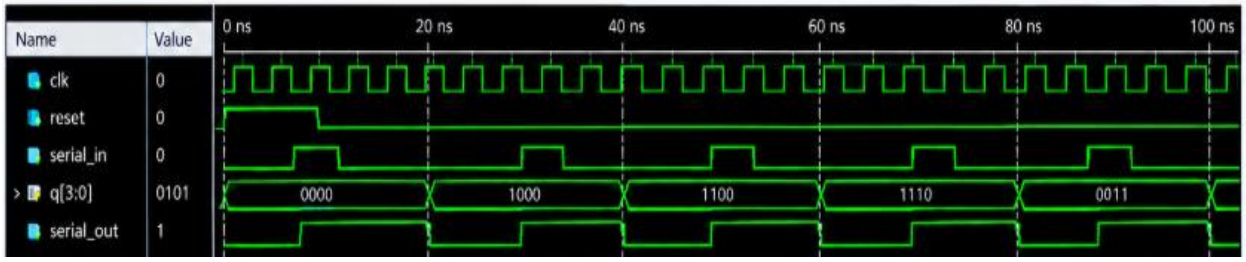
```
`timescale 1ns/1ps
module tb_shift_register4;
reg clk=0, reset=1, serial_in=0;
wire [3:0] q;
wire serial_out;
always #5 clk=~clk;
shift_register4 dut(.clk(clk),.reset(reset),.serial_in(serial_in),.q(q),.serial_out(serial_out));
initial begin
   #12; reset=0;
   serial_in=1; @(posedge clk); #1; $display("q=%b out=%b",q,serial_out);
   serial_in=0; @(posedge clk); #1; $display("q=%b out=%b",q,serial_out);
   serial_in=1; @(posedge clk); #1; $display("q=%b out=%b",q,serial_out);
   serial_in=1; @(posedge clk); #1; $display("q=%b out=%b",q,serial_out);
```

```
    serial_in=0; @(posedge clk); #1; $display("q=%b out=%b",q,serial_out);
    $finish;
end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



10. Divide-by-N Counter

Aim

Generate a divided clock output by toggling an output after a programmable number of input clock cycles.

Design Description

Parameter N is the number of input rising edges per output toggle. The output frequency is approximately f_in/(2N). For a divide-by-10 output frequency, set N=5. Reset is active high.

Control table:

reset	Counter condition	Behavior
1	X	count=0, clk_out=0
0	count < N−1	Increment count
0	count = N−1	Clear count and toggle clk_out

Verilog HDL Design Code

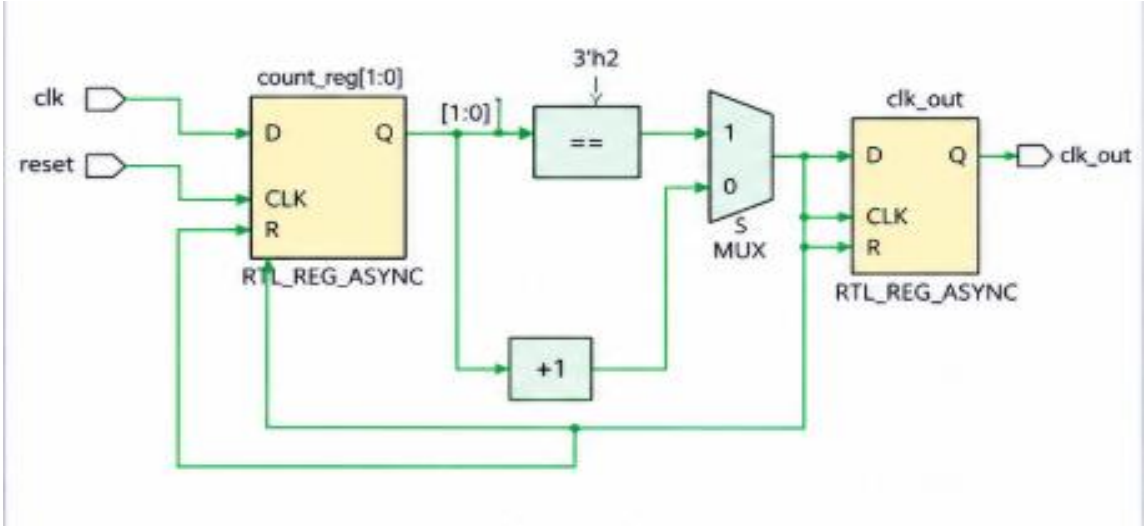
```
module divide_by_n #(
   parameter N = 5
) (
   input wire clk, reset,
   output reg clk_out
);
localparam CW = (N <= 1) ? 1 : $clog2(N);
reg [CW-1:0] count;
always @(posedge clk) begin
   if (reset) begin
      count <= 0;
      clk_out <= 0;
   end else if (count == N-1) begin
      count <= 0;
      clk_out <= ~clk_out;
   end else begin
      count <= count + 1'b1;
   end
end
endmodule
```

Testbench (TB)

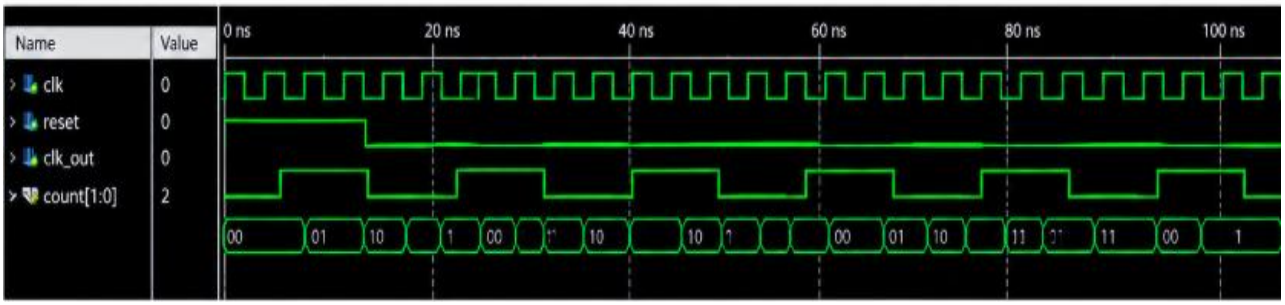
```
`timescale 1ns/1ps
module tb_divide_by_n;
```

```
reg clk=0, reset=1;
wire clk_out;
always #5 clk=~clk;
divide_by_n #(.N(3)) dut(.clk(clk),.reset(reset),.clk_out(clk_out));
initial begin
   #12; reset=0;
   repeat (24) begin @(posedge clk); #1; $display("time=%0t clk_out=%b",$time,clk_out); end
   $finish;
end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



11. Mealy and Moore FSMs

Aim

Implement a sequence detector for the serial pattern 101 using both Mealy and Moore finite-state machines.

Design Description

Input din is sampled on each rising clock edge. Mealy output asserts on the same cycle that the final 1 completes the sequence; Moore output asserts in a dedicated detected state one cycle after the transition that recognizes the sequence. Overlapping sequences are supported.

Sequence detector state/output summary for pattern 101:

FSM type	Output assertion
Mealy	Output is asserted when the final 1 completes 101
Moore	Output is asserted in a dedicated detected state after recognizing 101

Verilog HDL Design Code

```
module mealy_101 (
   input wire clk, reset, din,
   output reg detected
);
localparam S0=2'd0, S1=2'd1, S2=2'd2;
reg [1:0] state, next_state;
always @* begin
   next_state = state;
   detected = 1'b0;
   case (state)
      S0: if (din) next_state=S1; else next_state=S0;
      S1: if (din) next_state=S1; else next_state=S2;
      S2: if (din) begin next_state=S1; detected=1'b1; end
          else next_state=S0;
      default: next_state=S0;
   endcase
end
always @(posedge clk) begin
   if (reset) state <= S0;
   else       state <= next_state;
end
endmodule
```

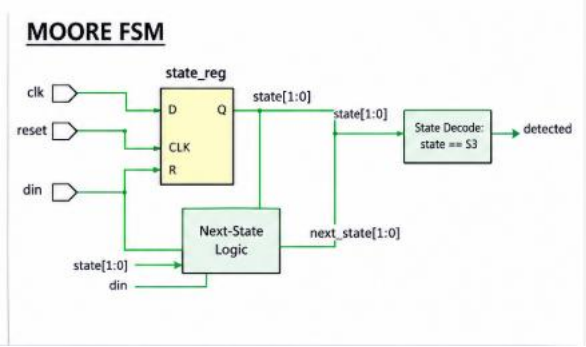
```
module moore_101 (
   input wire clk, reset, din,
   output wire detected
);
localparam S0=2'd0, S1=2'd1, S2=2'd2, S3=2'd3;
reg [1:0] state, next_state;
assign detected = (state == S3);
always @* begin
   next_state = state;
   case (state)
      S0: if (din) next_state=S1; else next_state=S0;
      S1: if (din) next_state=S1; else next_state=S2;
      S2: if (din) next_state=S3; else next_state=S0;
      S3: if (din) next_state=S1; else next_state=S2;
      default: next_state=S0;
   endcase
end
always @(posedge clk) begin
   if (reset) state <= S0;
   else       state <= next_state;
end
endmodule
```

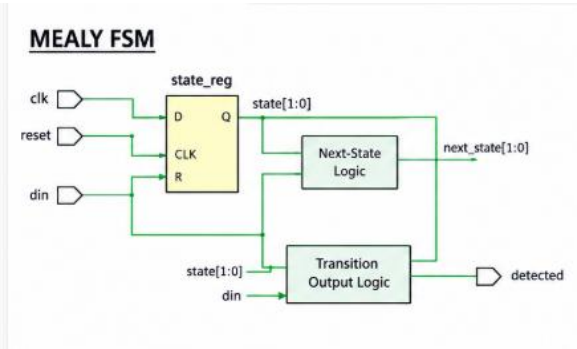
Testbench (TB)

```
`timescale 1ns/1ps
module tb_fsm_101;
reg clk=0, reset=1, din=0;
wire mealy_detected, moore_detected;
always #5 clk=~clk;
mealy_101 m0(.clk(clk),.reset(reset),.din(din),.detected(mealy_detected));
moore_101 r0(.clk(clk),.reset(reset),.din(din),.detected(moore_detected));
task send_bit(input reg b);
begin
   @(negedge clk); din=b;
   @(posedge clk); #1;
   $display("din=%b mealy=%b moore=%b",din,mealy_detected,moore_detected);
end
endtask
initial begin
   #12; reset=0;
   send_bit(1); send_bit(0); send_bit(1);
```

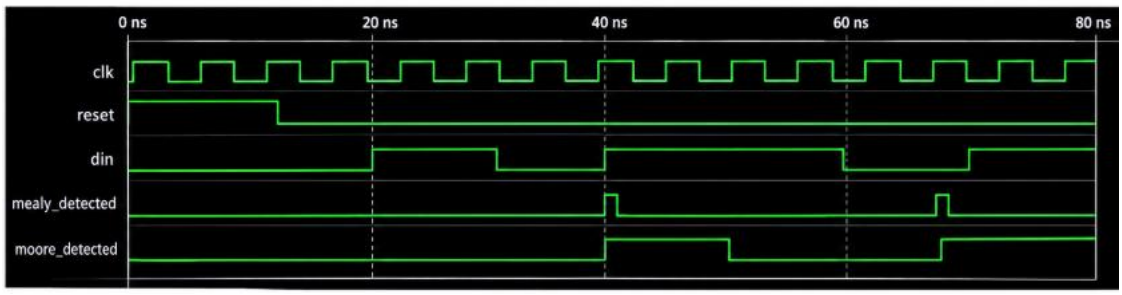
```
    send_bit(1); send_bit(0); send_bit(1);
    $finish;
end
endmodule
```

Circuit / RTL Schematic





Simulation Waveform / Output



12. Random Sequence Generator (LFSR example)

Aim

Generate a repeatable pseudo-random sequence using a 4-bit linear feedback shift register (LFSR).

Design Description

A 4-bit LFSR shifts each clock and uses XOR feedback from selected taps. Reset seed must be non-zero; an all-zero seed locks the LFSR. This example uses feedback q[3]^q[2] and seed 0001; sequence length depends on the chosen polynomial/tap convention.

LFSR transition table (sample states from the provided implementation):

Present state q[3:0]	Feedback q[3] XOR q[2]	Next state
0001	0	0010
0010	0	0100
0100	0	1000
1000	1	0001

Verilog HDL Design Code

```
module lfsr4(
   input clk, rst,
   output reg [3:0] q,
   output rnd
);
   always @(posedge clk or posedge rst) begin
      if (rst) q <= 4'b0001;              // non-zero seed
      else     q <= {q[2:0], q[3] ^ q[2]};
   end
   assign rnd = q[3];                    // serial random bit stream
endmodule
```

Testbench (TB)

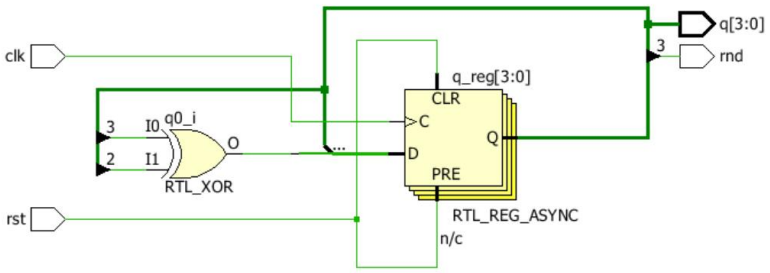
```
`timescale 1ns/1ps
module lfsr4_tb;
   reg clk = 0, rst = 1;
   wire [3:0] q;
   wire rnd;

   lfsr4 dut (.clk(clk), .rst(rst), .q(q), .rnd(rnd));
```

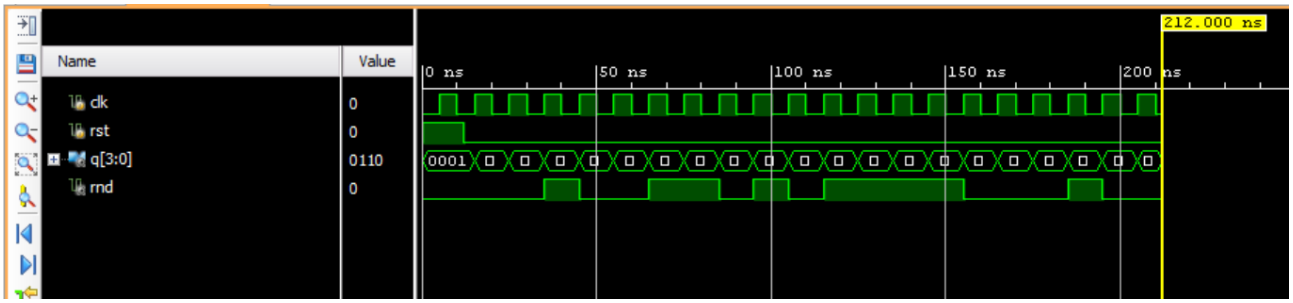
```
    always #5 clk = ~clk;

    initial begin
        $monitor("t=%0t q=%b (%d) rnd=%b", $time, q, q, rnd);
        #12 rst = 0;
        #200 $finish;                    // more than 15 clocks: sequence repeats
    end
endmodule
```

Circuit / RTL Schematic



Simulation Waveform / Output



Conclusion

The requested combinational and sequential digital modules have been described using Verilog HDL, with a corresponding testbench for each experiment. Behavioral simulation in Vivado can be used to verify the logic and timing of each design. The final submitted report should include the actual screenshots captured from the user's Vivado run and any observations required by the course instructor.

References

1. Xilinx, Vivado Design Suite User Guide: Logic Simulation (UG900).

2. Xilinx, Vivado Design Suite User Guide: Using Constraints (UG903).

3. Course notes and laboratory instructions for Verilog HDL and digital system design.